Aran Learn 19: Translating a Website (Tamil, Hindi, English)

For: everyone; translators especially. Arun and Bharadhwaj review the Hindi lines, Inbathamizhan and Aasif the Tamil lines. **Time:** about 1.5 hours. **Before this:** Learn 11 (HTML, CSS, JavaScript).

The console now speaks English, Tamil and Hindi (the switch is in the top bar). This doc explains how, and how we check that English is never broken.

1. Words you will hear

- **Internationalisation (i18n):** preparing the code so text can be swapped. (“i”, 18 letters, “n”.)
- **Localisation (l10n):** writing the actual Tamil or Hindi text.
- **Locale code:** a short name for a language: `en`, `ta`, `hi`; with a region, `ta-IN`.
- **Fallback:** what to show when a line is not translated. Ours is the English line.

2. How Aran does it: the English text is the key

`app/ui/js/i18n.js` holds one dictionary per language. The **English sentence itself is the key**:

```
ta: { 'A firewall for data': 'தரவுக்கான ஃபயர்வால்', ... }
hi: { 'A firewall for data': 'डेटा के लिए फ़ायरवॉल', ... }
```

Pages keep their English text where it always was. When they draw a heading they call `A.t(...)`:

```
A.t = (s) => (lang === 'en' ? s : T[lang][s] || s)
```

So:

- In English mode `A.t` returns its input untouched. **English cannot break**, because English is never looked up.
- A missing Tamil line shows the English line instead of a blank or a key like `tour.ch7.title`.

- The cost: if someone edits an English sentence, its translation silently stops matching. The check script below catches that.

The choice is remembered in `localStorage['aran-lang']`; changing it reloads the page. `<html lang="ta">` is set too, which tells screen readers which voice to use and lets CSS pick the Tamil font.

3. What is translated

The menu, the top bar, every page's eyebrow, title and lede, every tour chapter's name, title and tagline, and the tour buttons. The live scenes (log lines, tokens, tables) stay in English on purpose: they show real server data, and field names like `aadhaar=` must look exactly as the bank's systems write them.

4. Scripts and fonts

- Tamil and Hindi letters are often **several code points** that join into one visible letter (a consonant plus a vowel sign, like க + ◌ி = கி). Code that splits text with `[...text]` cuts them apart and shows dotted circles (◌◌ி). The rolling-letter links use `Intl.Segmenter`, which splits by **grapheme** (what a reader sees as one letter).
- Fonts: Noto Sans Tamil and Noto Sans Devanagari load from Google Fonts. Without internet the system's own fonts are used.
- Tamil words are long. Check headings at phone width; they must wrap, not overflow.

5. Checking it: `check-i18n.js`

```
node app/ui/tools/check-i18n.js
```

It loads the console scripts without a browser, collects every heading string, and reports:

```
English: 107/107 headings show their English text ✓
தமிழ் (ta): 107/107 translated ✓
हिन्दी (hi): 107/107 translated ✓
```

- **English** must always be complete; if not, the script exits with an error.
- **missing:** a heading with no translation (it falls back to English).
- **stale:** a translation whose English sentence no longer exists (someone changed the English). Copy the new English sentence into the key and update the translation.

Run it after every change to a page heading or tour chapter.

6. Translating well

- Translate the meaning, not word by word. “Harvest now, decrypt later” is an idiom; say it the way a Tamil or Hindi speaker would.
- Keep brand and technical names as they are: Aran, Aadhaar, PAN, UPI, OTP, ML-KEM, DPDP.
- Bank customers in India mix English words into Tamil and Hindi (டோக்கன், टोकन). Use the word people actually say.
- The drafts in `i18n.js` were written for review. Read every line aloud; fix anything that sounds unnatural.

Practice

1. Switch to Tamil, open each tour chapter at phone width, and note any heading that overflows.
2. Change one English page title in `pages.js`, run the check script, and watch it report the stale line. Fix it.
3. Add a fourth language (Telugu, `te`): one dictionary in `i18n.js`, one entry in `LANGS`, then run the check.